

PAWS & CART

Observability Case Study

React + Flask + SQLite • Prometheus + Grafana • Filebeat + Elasticsearch + Kibana

Field	Details
Course	Enterprise Software Development
Assignment	Assignment 1 — Observability
Student	Zainab Sadika Bhutto
Repository	petstore-observability
Submission Date	23 rd Sept 2026

FINAL REPORT

1. Project Overview

Paws & Cart is a small online pet-store application built to demonstrate end-to-end observability. The goal was not to build a complex e-commerce platform, but to create a working system whose application behaviour, business activity, infrastructure, logs, failures, and recovery could all be observed with real data.

The application uses a React/Vite frontend, a Flask REST API, and SQLite. The observability stack has two main pipelines: metrics flow from Flask and Node Exporter to Prometheus and then Grafana; structured JSON logs flow from Flask through Docker and Filebeat to Elasticsearch and Kibana. Docker Compose runs the complete stack.

1.1 Problem, Users and Working Solution

- Customers browse products, place orders, view current orders, and complete pending orders.
- Store operators can observe successful orders and the current pending-order backlog.
- Developers/operators can investigate traffic, latency, HTTP outcomes, host/container-environment resources, and individual requests.

Method	Endpoint	Purpose
GET	/health	Backend health check
GET	/products	List products
POST	/orders	Create an order
GET	/orders	List orders
POST	/orders/<id>/complete	Complete a pending order
GET	/metrics	Expose Prometheus metrics

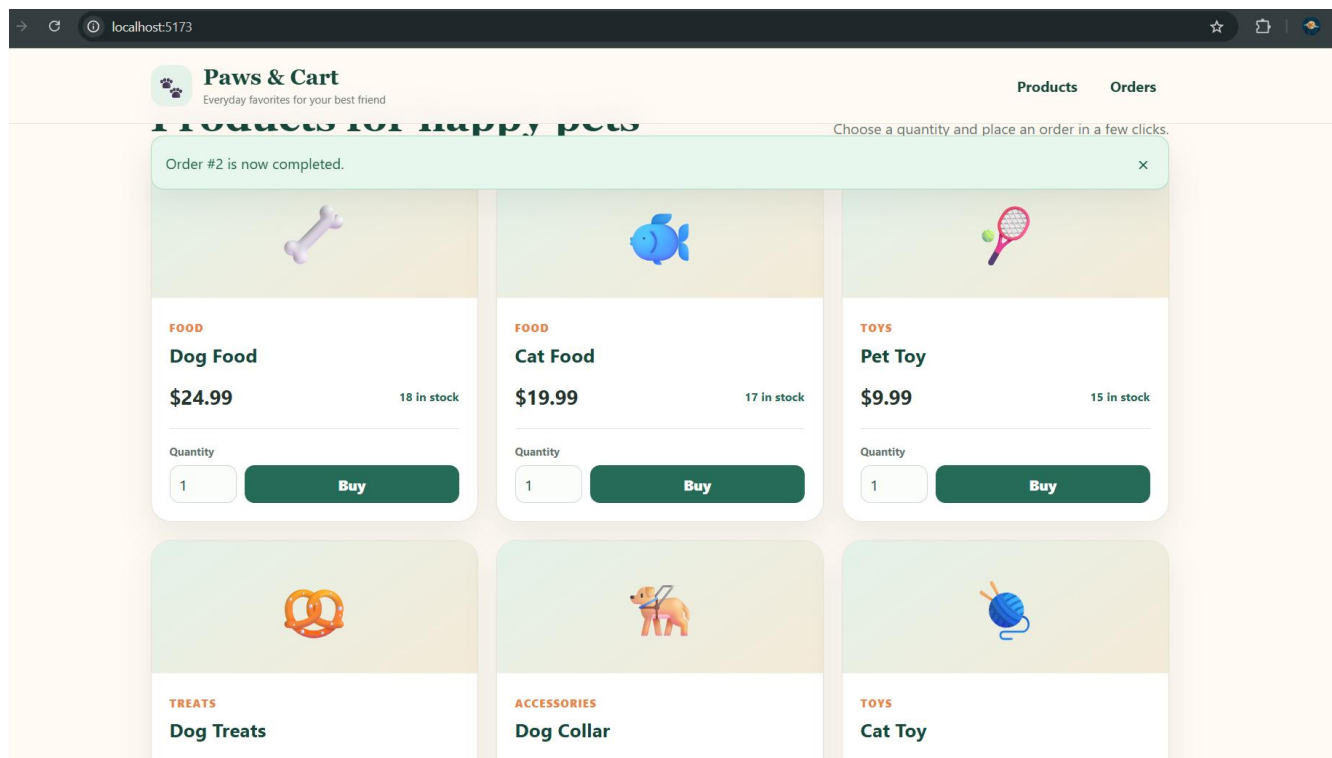


Figure 1. Paws & Cart frontend used to browse products and interact with the store.

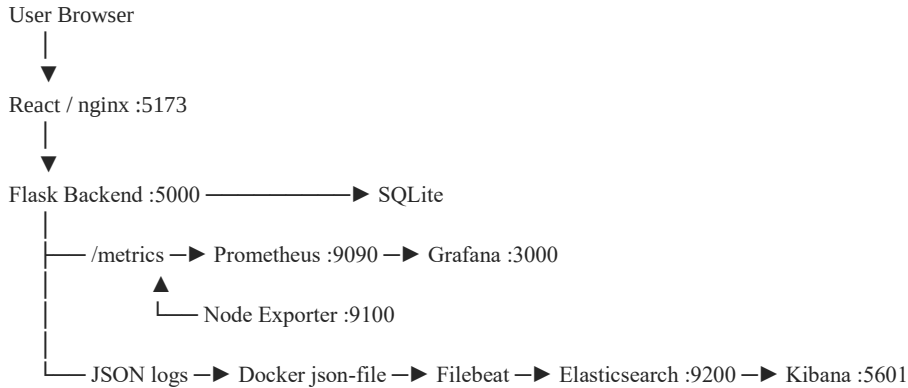
1.2 Technology Stack

Layer	Technology
Frontend	React + Vite, served through nginx
Backend	Python Flask
Database	SQLite
Metrics	Prometheus Python client + Prometheus
Dashboards	Grafana
Infrastructure metrics	Node Exporter
Logging	Structured JSON + Docker json-file

Log pipeline	Filebeat → Elasticsearch → Kibana
Deployment	Docker Compose

2. Architecture and Data Flow

The application remains usable independently of the visualisation tools. Prometheus scrapes metrics; Grafana reads from Prometheus. Separately, Docker captures backend JSON logs, Filebeat parses them, Elasticsearch stores them, and Kibana provides search and inspection.



Persistent Docker volumes are used for SQLite data, Prometheus data, Grafana data, and Elasticsearch data. One important exception is process-local Flask metric state: the order Counter resets when the Flask process restarts, while persistent SQLite order rows remain intact.

3. Metrics — Prometheus and Grafana

Prometheus scrapes the Flask /metrics endpoint approximately every five seconds and stores the samples as time series. Grafana queries Prometheus with PromQL; it does not read Flask directly. The project implements all four metric types required by the assignment.

Metric	Type	Purpose	Important labels / behaviour
petstore_orders_total	Counter	Successful order creations	Increments after DB commit; resets with Flask process
petstore_pending_orders	Gauge	Current pending orders	Reconstructed from SQLite after restart
petstore_http_request_duration_seconds	Histogram	HTTP latency and p95/p99	method, normalised route, status_code
petstore_order_processing_seconds	Summary	Successful order-processing time	sum/count used for average

3.1 Key Metric Design Decisions

Dynamic routes are normalised before becoming labels. For example, individual order IDs are represented by /orders/<int:order_id>/complete instead of creating a new label value for every order. This keeps metric cardinality bounded. Prometheus's own /metrics traffic is excluded from application-traffic dashboard queries so monitoring activity is not mistaken for user traffic.

The Python Prometheus Summary implementation used here provides `_sum` and `_count`, which are used for average processing time. Tail latency is therefore calculated from the Histogram with `histogram_quantile()`, not from the Summary.

3.2 Important Grafana Queries

Panel	PromQL / interpretation
Pending orders	petstore_pending_orders
Orders over time	increase(petstore_orders_total[5m])
Average processing time	1000 * rate(petstore_order_processing_seconds_sum[5m]) / rate(petstore_order_processing_seconds_count[5m])
Request rate	sum(rate(petstore_http_request_duration_seconds_count{route!="/metrics"}[5m]))
Request rate by route	sum by (route) (rate(petstore_http_request_duration_seconds_count{route!="/metrics"}[5m]))
p95 / p99	1000 * histogram_quantile(0.95 or 0.99, sum by (le) (rate(..._bucket{route!="/metrics"}[5m])))

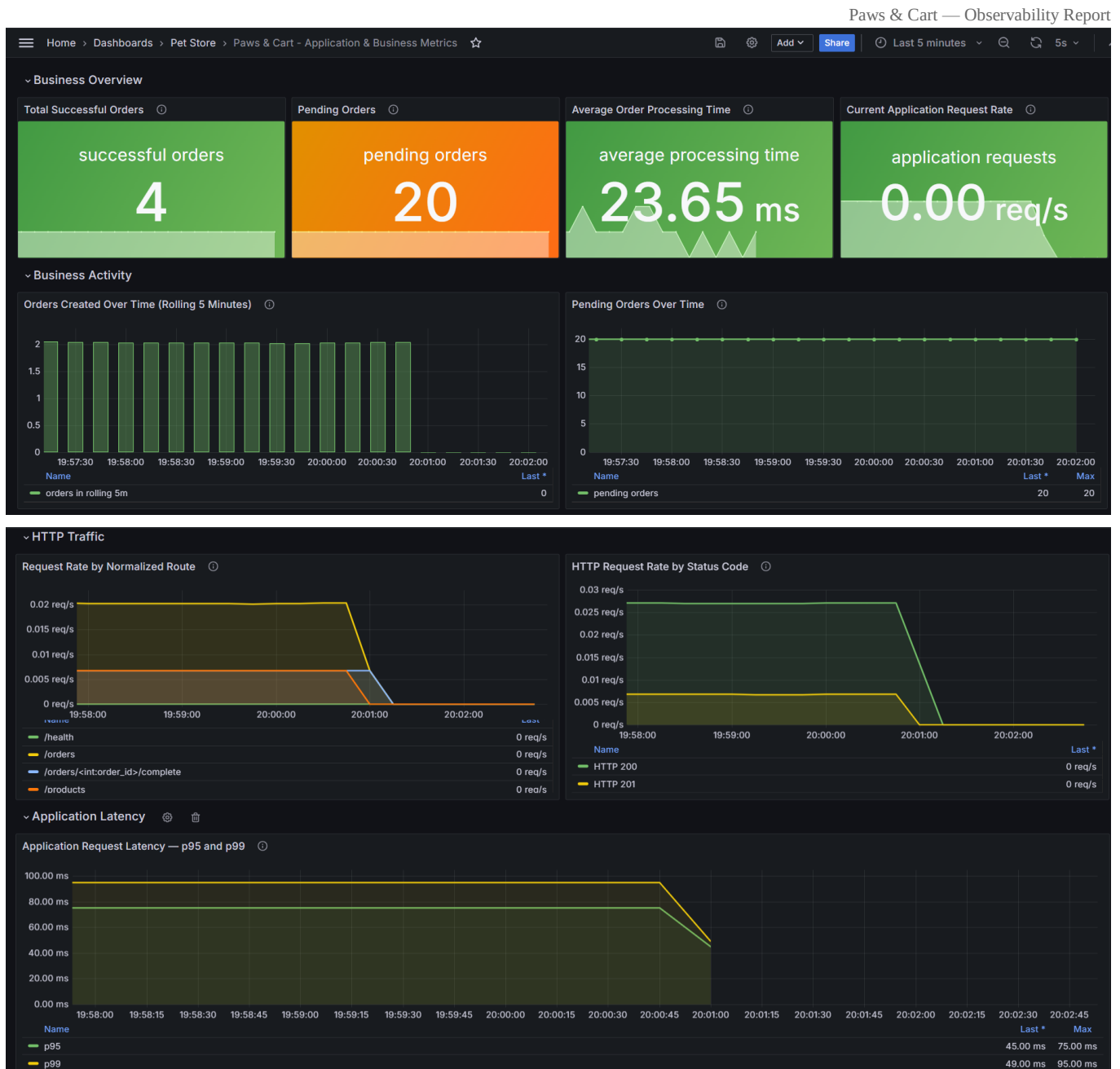
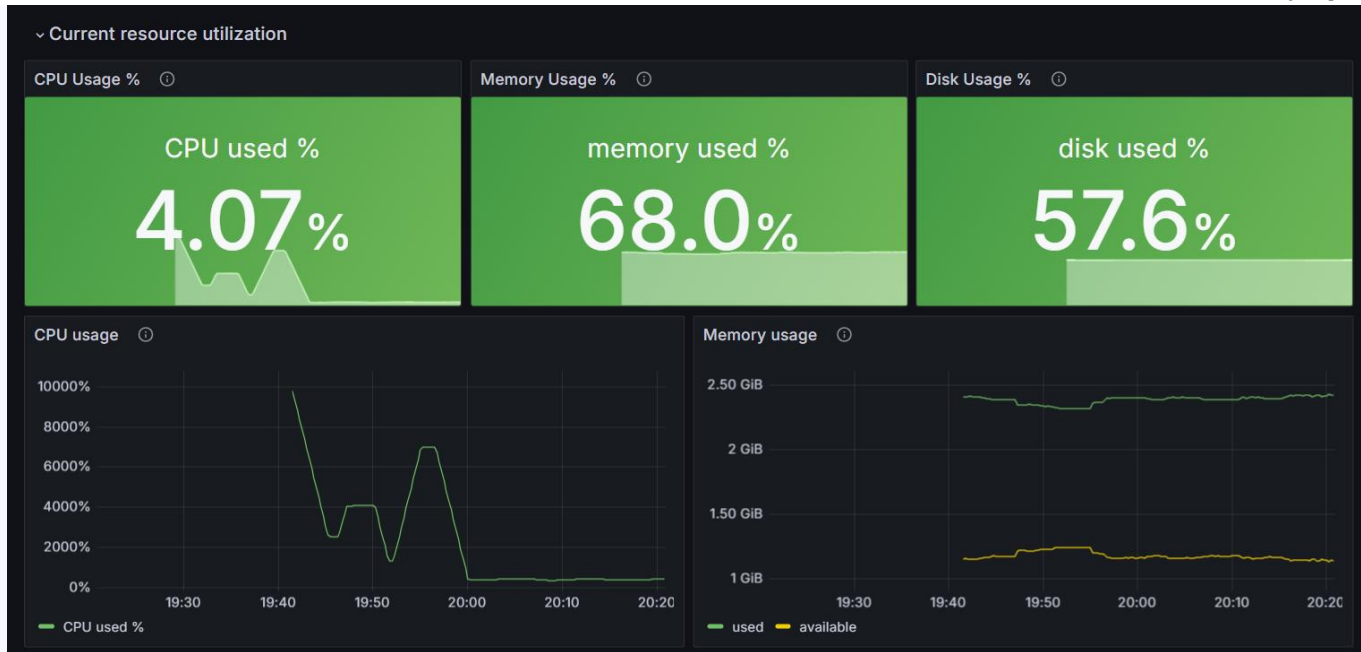


Figure 2. Grafana application/business dashboard showing business KPIs, route traffic, and p95/p99 latency.

Above figures show the business and application metrics collected by Prometheus and visualized in Grafana. The dashboard tracks successful and pending orders, average processing time, request rate, traffic by normalized route and HTTP status code, and application latency. At the time of capture, the dashboard showed 4 successful orders, 20 pending orders, a 23.65 ms average processing time, and a request rate of 0.00 req/s. The latency Histogram is also used to calculate p95 and p99 over a rolling five-minute window, which were approximately 75 ms and 95 ms respectively. Dashboard values change as the application receives and processes requests.

3.3 Node Exporter

Node Exporter supplies CPU, memory, disk, and network metrics to Prometheus. Because the stack runs through Docker Desktop on Windows, the accurate claim is that Node Exporter monitors the Linux environment used by Docker Desktop to host the containers, not the physical Windows host directly.



4. Structured Logging — Filebeat, Elasticsearch and Kibana

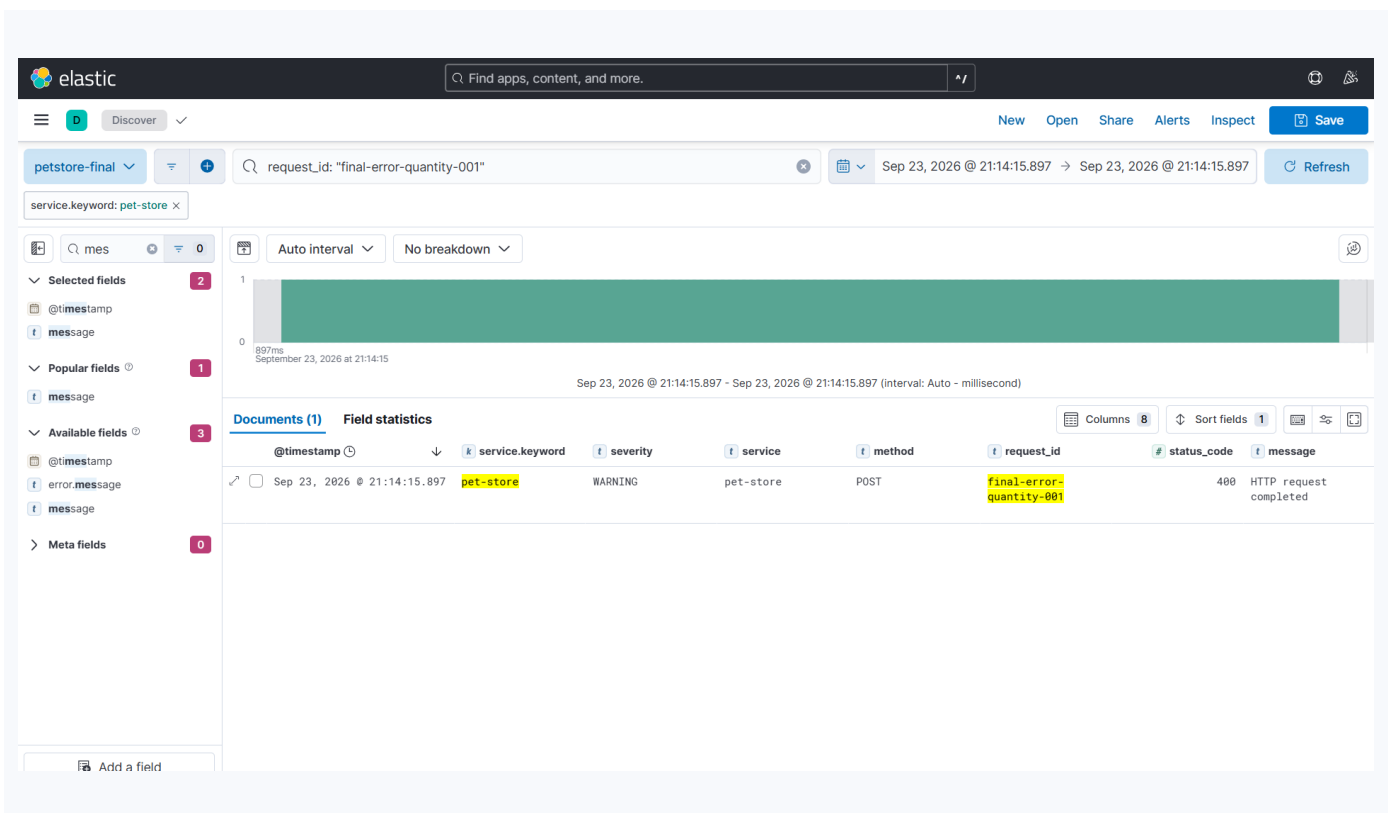
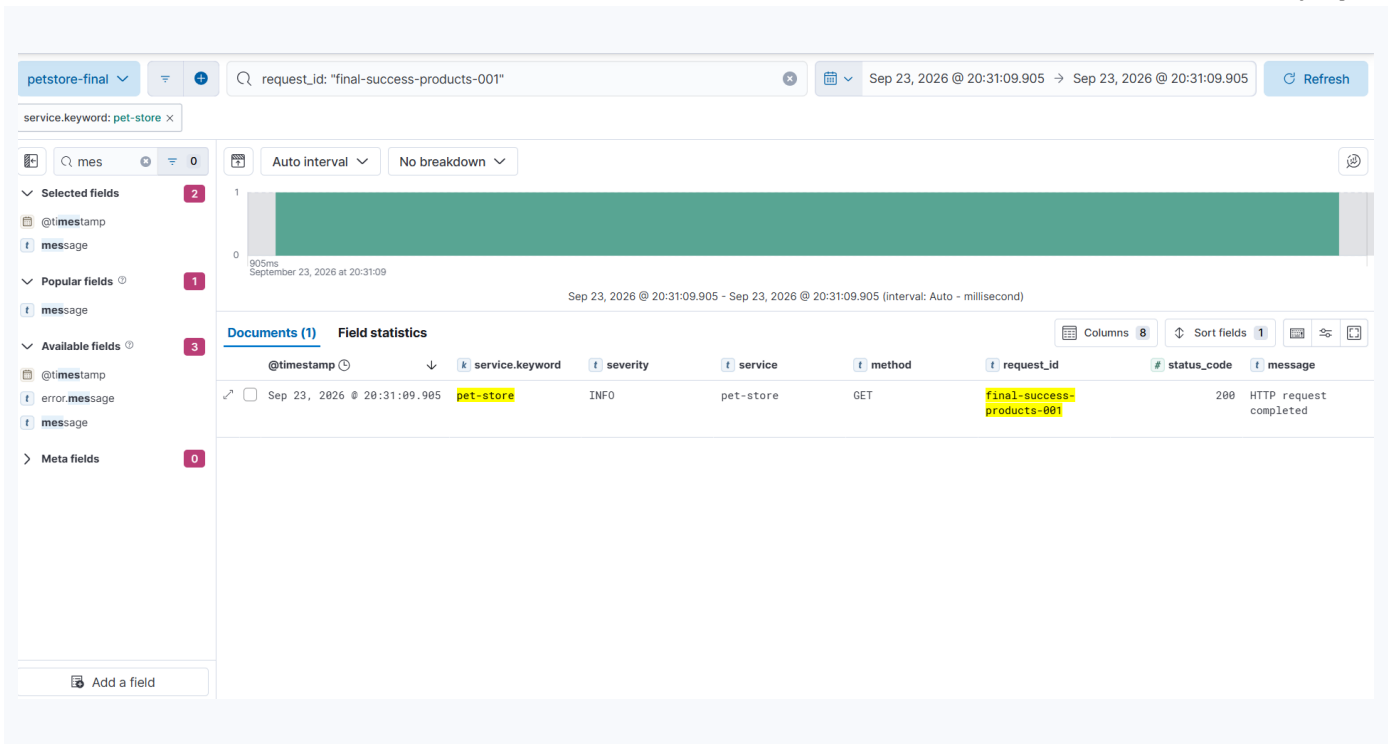
Metrics answer aggregate questions such as whether latency is increasing; structured logs answer request-specific questions. The logging pipeline is Flask → Docker → Filebeat → Elasticsearch → Kibana.

Field	Meaning
time	Original Flask event time
service	Service name (pet-store)
severity	INFO / WARNING / ERROR
message	Human-readable event description
request_id	Correlation identifier
method / route	HTTP operation and normalised route
status_code	HTTP outcome
duration_ms	Measured request duration

If an incoming request provides X-Request-ID, Flask reuses it; otherwise a UUID is generated. The same identifier is returned in the response header. Severity is derived from status: responses below 400 are INFO, 4xx responses are WARNING, and 5xx responses are ERROR.

Docker uses the json-file logging driver with rotation (approximately 10 MB per file and three files). Filebeat watches Docker, selects the backend container using its Docker label, decodes the application's JSON message into searchable fields, and sends events to Elasticsearch.

The project uses the regular Elasticsearch index petstore-final with data persisted in the elasticsearch_data volume. Explicit ILM is disabled, so no automatic Elasticsearch retention period is configured. Kibana uses the petstore-final* data view with @timestamp as the time field.



5. End-to-End Trace and Failure Behaviour

5.1 Metric Trace

Successful order → SQLite transaction commits → `petstore_orders_total.inc()` → `/metrics` exposes the updated Counter → Prometheus scrapes and stores the sample → Grafana queries Prometheus and visualises the result.

5.2 Log Trace

HTTP request → request ID accepted/generated → Flask after-request hook measures method, normalised route, status and duration → structured JSON emitted → Docker captures it → Filebeat parses it → Elasticsearch indexes it → Kibana searches it by request_id.

5.3 Failure Behaviour

Component unavailable	Expected effect
Grafana	Application and Prometheus continue; only visualisation is unavailable.
Prometheus	Flask continues and /metrics remains available, but missed samples are not reconstructed exactly.
Filebeat	Flask and Docker logging continue; new events are not delivered to Elasticsearch while Filebeat is unavailable.
Elasticsearch	Application continues; Filebeat cannot successfully deliver indexed events.
Kibana	Elasticsearch/Filebeat continue; only the search interface is unavailable.

6. Controlled Experiment 1 — Latency Anomaly

The experiment followed the required cycle: establish normal behaviour, predict the effect, introduce a reproducible fault, observe it, remove the fault, and retest. The backend flag PETSTORE_INJECT_DELAY adds approximately 500 ms to every fifth Flask request when enabled.

A dedicated PowerShell workload sends only GET /products requests, so the experiment does not create orders, consume stock, or modify SQLite business data.

```
.\part-e\anomaly_workload.ps1 -Scenario <baseline|anomaly|recovery> -RequestCount 50
```

Measurement	Baseline	Anomaly	Recovery
Successful requests	50/50	50/50	50/50
Average duration	16.013 ms	117.949 ms	12.665 ms
Maximum duration	109.913 ms	516.253 ms	28.446 ms
Requests ≥ 450 ms	0	10	0

The anomaly produced 10 slow requests out of 50 (20%), matching the intended one-in-five pattern. After the flag was restored to false, the workload returned to normal with no requests above 450 ms. This demonstrates that the fault was reproducible and reversible.

```
Completed scenario 'baseline'.
Successful requests: 50/50
Average duration: 16.013 ms
Maximum duration: 109.913 ms
Requests at or above 450 ms: 0
```

```
Completed scenario 'anomaly'.
Successful requests: 50/50
Average duration: 117.949 ms
Maximum duration: 516.253 ms
Requests at or above 450 ms: 10
```

```
Completed scenario 'recovery'.
Successful requests: 50/50
Average duration: 12.665 ms
Maximum duration: 28.446 ms
Requests at or above 450 ms: 0
```

6.1 Grafana p95/p99 Evidence

During the anomaly, the Grafana latency panel showed a maximum p95 of approximately 887.50 ms and a maximum p99 of approximately 977.50 ms. These values are histogram bucket-based percentile estimates over a rolling five-minute window, rather than direct single-request maxima. The workload itself recorded a maximum request duration of 590.429 ms, so the Grafana percentile estimates and workload timing should not be interpreted as the same measurement.



7. Controlled Experiment 2 — Prometheus Label Cardinality

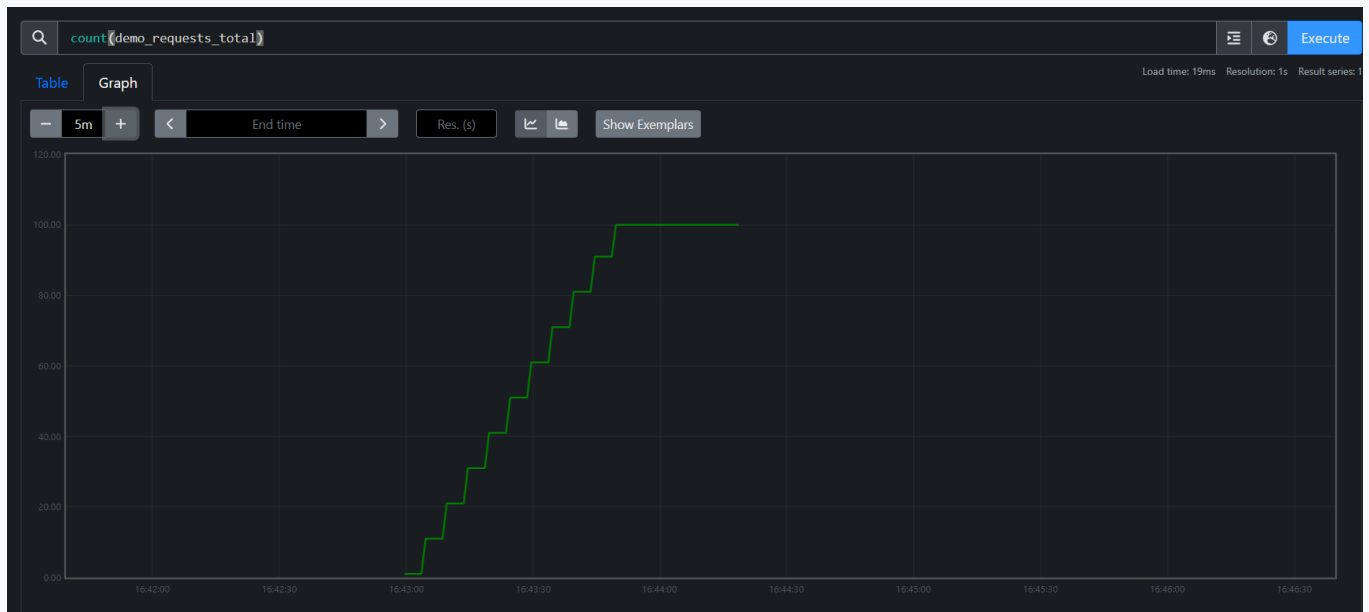
The second experiment demonstrates why unbounded values such as request IDs should not be Prometheus labels. The standalone part-e/cardinality_demo.py exposes demo_requests_total on port 8010.

7.1 BAD_LABEL=true

The bad version attaches a unique request_id UUID to every request. One hundred requests therefore create one hundred distinct label sets/time series. Prometheus was queried with:

```
count(demo_requests_total)
```

Observed result: the graph reached 100 series while the BAD demo was active. This is direct evidence of high cardinality caused by an effectively unbounded label.

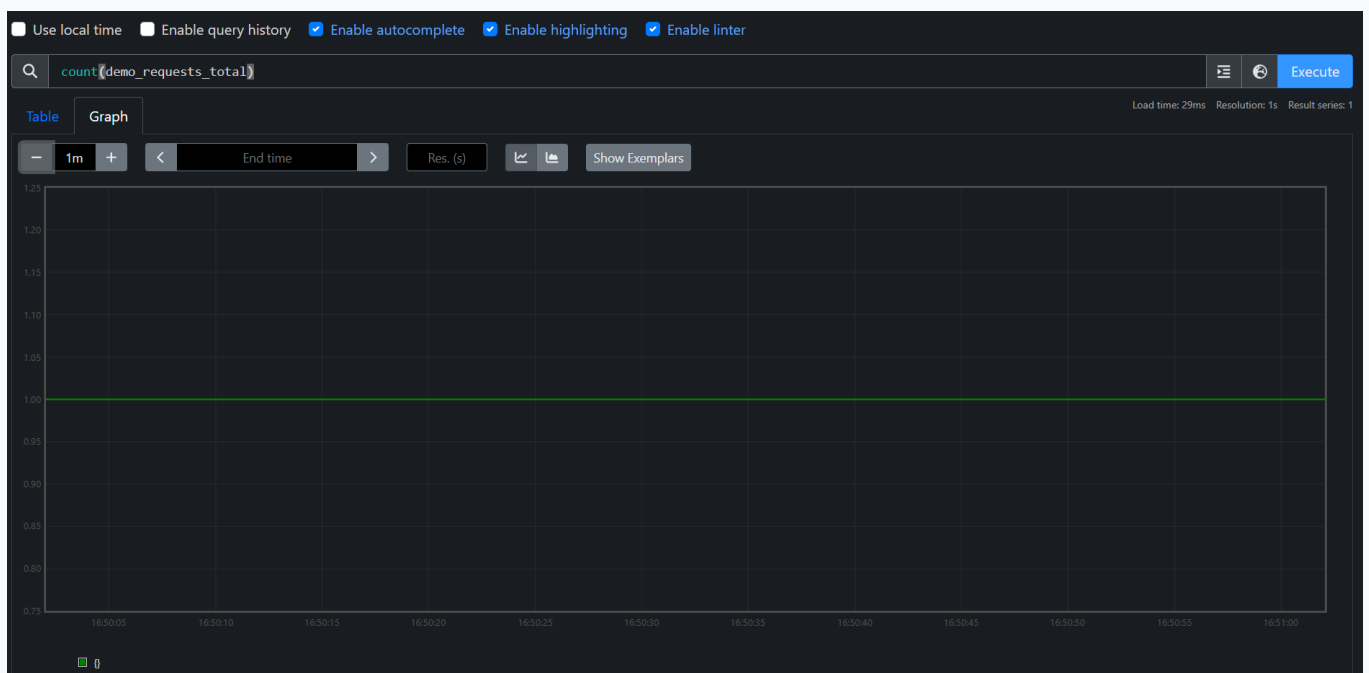


7.2 BAD_LABEL=false

The good version removes `request_id` from the metric label. The same 100 requests share one label set, so the active series count is 1.

`count(demo_requests_total)`

Observed result: the later graph dropped to 1 series for the GOOD implementation. The temporary demo target becomes DOWN after the short-lived container exits, which can make a later instant query empty; the graph history preserves the experimental evidence.



7.3 Interpretation

Prometheus labels should use bounded dimensions such as method, route, and status_code. A request ID is effectively unbounded; at large traffic volumes it can create enormous numbers of time series and increase memory, indexing, storage, and query cost. Request IDs therefore belong in structured logs, where Elasticsearch and Kibana can search them efficiently. This experiment directly connects the metrics and logging design decisions in the project.

8. Key Technical Decisions and Final Status

- Normalised route labels prevent individual order IDs from creating unnecessary Prometheus series.
- The /metrics route is excluded from user-traffic dashboards.
- The pending-order Gauge is reconstructed from SQLite after backend restart.
- Histogram buckets provide p95/p99; Summary sum/count provides average order-processing time.
- Request IDs are searchable log fields, not Prometheus labels.
- Filebeat is scoped to the backend container rather than unrelated Docker containers.
- Docker log rotation limits raw log growth; Elasticsearch data persists in a Docker volume.
- The anomaly workload is read-only and does not alter store inventory or order state.

8.1 Completion Status

Requirement	Status
Working pet-store application	Complete
Prometheus + all four metric types	Complete
Grafana application/business dashboards	Complete
p95/p99 latency and rolling-window explanation	Complete
Node Exporter CPU/memory/disk/network	Complete
Structured JSON logging	Complete
Filebeat → Elasticsearch → Kibana	Complete
Architecture, traces and failure behaviour	Complete
Latency anomaly experiment	Complete
Cardinality BAD vs GOOD experiment	Complete — observed 100 → 1
README / reproducibility documentation	Complete

9. Conclusion

Paws & Cart demonstrates a complete observability workflow around a deliberately small application. Prometheus and Grafana provide aggregate business, traffic, latency, and infrastructure visibility, while Filebeat, Elasticsearch, and Kibana provide request-level searchable evidence. The latency experiment showed that a controlled fault could be predicted, observed in raw timings and percentile dashboards, removed, and verified as recovered. The cardinality experiment showed why request IDs belong in logs rather than metric labels by changing the same demonstration from 100 series to 1. Together, these results show not only that the tools were installed, but that their design trade-offs and operational behaviour were understood and tested.